The Mathematics of tagd

A Formal Description of the Mathematical Structures
in the tagd Semantic-Relational System

Isaac N Colley

September 2026

"To be, is to be related."
— tagd Maxim

Abstract

The tagd system was built from a vision of a hierarchical structure of knowledge that is fully connected and consistent within itself. This paper makes the mathematics of that vision explicit. Its central thesis is that *information is structural*: in tagd, meaning is defined through relations. A tag is situated by its subordinate relation and described by its predicates. Rank strings encode this hierarchy as a partial order, whose Alexandrov topology gives precise form to containment, neighbourhoods, and continuity. The paper develops these connections and explores how they can support a relational account of computation.

Contents

1 Primitives **2**
- 1.1 Tags . . . 2
- 1.2 The Root and the Axiom . . . 3

2 The Subordinate Relation and the Rooted Tree **3**
- 2.1 Subordinate Relations . . . 3
- 2.2 The Tagspace as a Rooted Tree . . . 4

3 Rank: Encoding the Tree in Strings **4**
- 3.1 The Rank Function . . . 4
- 3.2 The Prefix Partial Order . . . 5
- 3.3 Verifying the Partial Order Axioms . . . 5
- 3.4 Subtrees and Their Intersection . . . 5
- 3.5 Lexicographic Total Order . . . 6
- 3.6 CS Tree versus Topological Poset: Orientation . . . 6

4 The Tagspace Topology **6**
- 4.1 From Partial Order to Topology . . . 6
- 4.2 Verifying the Topology Axioms . . . 7
- 4.3 Specialization Order . . . 7
- 4.4 Continuity as Order-Preservation . . . 7

5 The Hard Tag Base: A Common Subspace **8**
- 5.1 Hard Tags . . . 8
- 5.2 The Hard Tag Subspace . . . 8
- 5.3 Interoperability via the Common Base . . . 8

6 Predicates: Fibered Structure **9**
6.1 Relators Are Tags . . . 9
6.2 Predicates as Horizontal Relations . . . 9
6.3 The Two-Layer Structure . . . 9
6.4 Query Semantics as Selection of Sets . . . 10

7 Structural Identity **11**
7.1 Identity Is Position . . . 11
7.2 The Canonical Form . . . 11

8 Deeper Structures **11**
8.1 The Adjunction Between Order and Topology . . . 11
8.2 Topological Properties of the Tagspace . . . 12
8.3 Modal Operators: Interior and Closure . . . 12
8.4 Constructive Existence: Tags as Witnesses . . . 13

9 Planned Future Developments **13**
9.1 Completing the Structural Operations . . . 13
9.2 Kripke Semantics and Modal Queries . . . 14
9.3 Martin-Löf Type Theory and Dependent Structure . . . 14
9.4 Computation as Topological Traversal . . . 14
9.5 Signed Merkle Versions of Tagspaces . . . 14

10 Summary **15**

11 Conclusion **15**

1 Primitives

1.1 Tags

A **tag** is the atomic unit of the tagd system: a node in the hierarchy and a point in the tagspace topology developed below. Tag identifiers are UTF-8 labels [9]. A tag carries two distinct identities:

- **Nominal identity**: a UTF-8 label unique within a tagspace.
- **Structural identity**: its position in the tagspace hierarchy, given by its rank, defined by its subordinate relation (aka identity relation) [7].

Definition 1.1 (Tag Set)**.** *Let T be the set of all tags in a tagspace. Each tag $t \in T$ is identified by a string* $\mathrm{id}(t) \in \Sigma^*$*, where Σ^* denotes the set of all finite UTF-8 strings.*

In TAGL, defining a tag looks like:

```
>> dog _is_a mammal;
```

Listing 1: Defining a tag via a subordinate relation

Here `dog` is a tag. Its nominal identity is the string `"dog"`. Its structural identity is defined by its subordinate relation to `mammal`.

1.2 The Root and the Axiom

Every tagspace has a distinguished root tag: `_entity`. The README states:

> `_entity` is axiomatic and self-referencing (i.e. `_entity _sub _entity`). It is the only tag having this relation. It is the root of a tagspace.

Definition 1.2 (Axiomatic Root)**.** *The tag* ***`_entity`*** *is the* ***axiomatic root*** *of every tagspace. It is the unique fixed point of the parent function extended to the root: the one tag whose super-object is itself.*

In TAGL notation:

```
_entity _sub _entity
```

Listing 2: The axiomatic root

The symbolic operator `<:` maps to `_sub` exactly.

```
_entity <: _entity
```

Listing 3: TAGL subordinate symbol

In hierarchical terms, `_entity` is the **root** of the tagspace — the tag that is an ancestor of everything, including itself.

2 The Subordinate Relation and the Rooted Tree

2.1 Subordinate Relations

A tag's structural identity is defined by one subordinate (sub) relation: a sub-relator and a parent [7, 8]. The code represents these as `sub_relator` and `super_object`.

The sub relators are defined in `hard-tags.h` [1]:

```
inline constexpr std::string_view HARD_TAG_SUB{"_sub"}; /// HARD_TAG_SUB
    HARD_TAG_ENTITY tagd::POS_SUB_RELATOR
// Leibniz discernment from _sub will come as constraints in TAGL are developed
inline constexpr std::string_view HARD_TAG_IS_A{"_is_a"}; /// HARD_TAG_SUB
    HARD_TAG_SUB tagd::POS_SUB_RELATOR
inline constexpr std::string_view HARD_TAG_TYPE_OF{"_type_of"}; /// HARD_TAG_SUB
    HARD_TAG_SUB tagd::POS_SUB_RELATOR
```

Listing 4: Sub relator definitions in hard-tags.h

A typical TAGL knowledge hierarchy:

```
>> physical_object _sub _entity;
>> living_thing _type_of physical_object;
>> animal _type_of living_thing;
>> mammal _type_of animal;
>> dog _is_a mammal;
```

Listing 5: Building a subordinate hierarchy in TAGL

Remark 2.1 (Direct subordination versus containment)**.** *A subordinate relation records the pair* $(s, p(t))$*, where* s *is its sub-relator and* $p(t)$ *its immediate parent. This edge is distinct from the reflexive, transitive ancestor relation encoded by rank prefixes. For a non-root tag, the parent is its unique lower cover in the ancestor-first order used below; the root has no lower cover.*

2.2 The Tagspace as a Rooted Tree

The subordinate relation organizes tags into a **rooted tree**.

Definition 2.1 (Tagspace Tree)**.** *A tagspace is a pair (T, p) where:*

- *T is a set of tags containing the distinguished root `_entity`,*
- *$p : T \setminus \{\texttt{_entity}\} \to T$ is the* parent function*, assigning each non-root tag its super-object.*

Every non-root tag reaches `_entity` after finitely many parent steps. There are no cycles (except the axiomatic self-reference of `_entity`, when p is extended to the root).

The tree structure of the example above:

```
_entity
  physical_object
    living_thing
      animal
        mammal
          dog
```

The tree expresses hierarchical knowledge: `dog` is related to `mammal`, which is related to `animal`, and so on. Each parent step contributes to the tag's structural identity.

> **Future development**
> Here `_is_a` and `_type_of` both establish subordination; their finer logical interpretation is discussed in Section 9.

3 Rank: Encoding the Tree in Strings

3.1 The Rank Function

Each tag is assigned a **rank** — a UTF-8 byte string encoding its position in the tagspace tree. The `rank.h` source states:

> A rank string is the ordinal value of each node in the tagspace tree. A code point at a particular level is known as the rank value.

Definition 3.1 (Rank Function)**.** *The rank function $r : T \to \Sigma^*$ assigns to each tag a UTF-8 string such that r is injective and a child's rank extends its parent's rank by one code point. The root has the empty rank.*

For example, the following hard tags have these ranks. Dots separate the code points along a path; the stored rank is a byte string [10]:

Tag	Rank (dotted)	Position
`_entity`	(empty)	level 0, root
`_sub`	1	level 1
`_is_a`	1.1	level 2
`_type_of`	1.2	level 2
`_rel`	2	level 1
`_has`	2.1	level 2
`_true`	3	level 1
`_false`	4	level 1
`_interrogator`	5	level 1

3.2 The Prefix Partial Order

The key property of ranks is stated explicitly in `rank.h`:

> All children of a particular node in the rank hierarchy will be prefixed by the parent node — that is, rank substrings define subtrees in the hierarchy.

Definition 3.2 (Containership Relation)**.** *Define the* ***containership relation*** $\sqsubseteq$ *on* T *by:*

$$a \sqsubseteq b \iff r(a) \text{ is a prefix of } r(b).$$

That is, $a \sqsubseteq b$ *means* a *is an ancestor of* b *(or* $a = b$*). Tag* a contains *tag* b *in the hierarchy.*

For nonempty ranks, this prefix test is declared in `rank.h` and implemented by `rank::contains()` in `rank.cc`:

```
// this rank contains the rank being compared,
// in other words, this rank is a prefix to the other
bool contains(const rank&) const;
```

Listing 6: Containership via rank prefix in rank.h

> **Implementation note**
>
> The root is represented by the empty prefix. The tagspace-level containment API handles it explicitly; `rank::contains()` compares nonempty ranks.

The nonempty-rank test is used in `tagd.cc` for structural operations on tag sets:

```
if (b->rank().contains(a->rank())) { // rank can contain itself (b == a)
    merged++;
} else if (a->rank().contains(b->rank())) {
    A.insert(*b);
}
```

Listing 7: Containership used in merge_containing_tags (tagd.cc)

3.3 Verifying the Partial Order Axioms

Theorem 3.1. *The containership relation* $\sqsubseteq$ *is a partial order on* T*.*

Proof. We verify the three required properties.

Reflexivity. Every string, including the empty string, is a prefix of itself. Hence $a \sqsubseteq a$ for all $a \in T$.

Antisymmetry. Suppose $a \sqsubseteq b$ and $b \sqsubseteq a$. Then $r(a)$ is a prefix of $r(b)$ and $r(b)$ is a prefix of $r(a)$. A string u is a prefix of v and v is a prefix of u if and only if $u = v$. Hence $r(a) = r(b)$, which implies $a = b$ since ranks uniquely identify a tag's position in a tagspace.

Transitivity. Suppose $a \sqsubseteq b$ and $b \sqsubseteq c$. Then $r(a)$ is a prefix of $r(b)$ and $r(b)$ is a prefix of $r(c)$. Since the prefix relation on strings is transitive, $r(a)$ is a prefix of $r(c)$, so $a \sqsubseteq c$. □

Therefore $(T, \sqsubseteq)$ is a **poset** (partially ordered set).

3.4 Subtrees and Their Intersection

For $x \in T$, define the descendant subtree, including its root, by

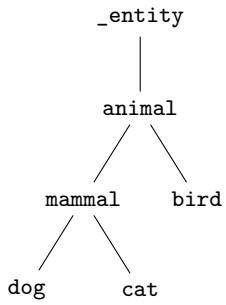
$$D(x) = \{t \in T : x \sqsubseteq t\}.$$

The unique-parent property makes this family *laminar*: two subtrees are nested or disjoint. Explicitly,

$$D(x) \cap D(y) = \begin{cases} D(y), & x \sqsubseteq y, \\ D(x), & y \sqsubseteq x, \\ \varnothing, & \text{otherwise.} \end{cases}$$

If a tag belonged to both subtrees, both roots would lie on its unique ancestor chain and hence would be comparable. This proves the last case; the first two follow directly from transitivity.

For example, with `dog` and `cat` subordinate to `mammal`, and `bird` on a separate branch subordinate to `animal`, $D(\texttt{animal}) \cap D(\texttt{mammal}) = D(\texttt{mammal})$, whereas $D(\texttt{mammal}) \cap D(\texttt{bird}) = \varnothing$. This is the set-theoretic basis for retaining the more specific root when intersecting subtree selections. A descendant subtree does not include its root's strict ancestors; shared ancestry therefore does not change the empty-intersection case. Common ground between entire tagspaces is considered in Section 5.3.



3.5 Lexicographic Total Order

Ranks also carry a total **lexicographic order**, implemented in `rank.h`:

```
bool operator<(const rank& rhs) const { return _data < rhs._data; }
```

Listing 8: Lexicographic ordering of ranks

This order — a deliberate UTF-8 design choice noted in `rank.h` ("it collates lexicographically") — enables efficient storage and retrieval of ranks as a sorted structure. Tags within the same subtree sort together. The hierarchy is therefore encoded directly in the ordering of the index.

> **Implementation note**
> SQLite uses `COLLATE BINARY` for the rank column and `ORDER BY rank` in `.dump_grid` [5].

3.6 CS Tree versus Topological Poset: Orientation

A tree drawing places `_entity` at the top, but the ancestor-first prefix order makes it the **least element** ($\bot$): its empty rank prefixes every rank and has the lowest value under binary lexicographic collation using C++'s less-than operator `<`. This total order extends the prefix partial order; it also compares unrelated branches. Leaves are **maximal elements** of the partial order, not necessarily greatest in lexicographic order.

4 The Tagspace Topology

4.1 From Partial Order to Topology

We choose the upper-set Alexandrov topology of the ancestor-first poset $(T, \sqsubseteq)$ [6]. This makes the tagspace a topological space whose basic open sets are its subtrees:

Definition 4.1 (Tagspace Open Sets)**.** *A subset $U \subseteq T$ is **open** if it is upward-closed under $\sqsubseteq$:*

$$U \in \mathcal{T} \iff \forall\, a \in U,\ \forall\, b \in T :\ a \sqsubseteq b \Rightarrow b \in U.$$

In the tagspace tree, "upward-closed" means: if a tag is in the set, all its descendants are too. The smallest open neighbourhood of x is its subtree

$$N(x) = \{b \in T : x \sqsubseteq b\} = D(x) = \uparrow x.$$

Each $D(x)$ is a principal filter; the closure of $\{x\}$ is its ancestry, the principal ideal $\downarrow x = \{b \in T : b \sqsubseteq x\}$. There is also a dual Alexandrov topology of downward-closed sets, in which $D(x)$ is a closed set. Both topologies arise from the same tree; this paper keeps the upward-open convention unless explicitly stated otherwise.

This construction is known as the **Alexandrov topology** — the topology generated by a partial order in which arbitrary intersections (not just finite ones) of open sets remain open.

4.2 Verifying the Topology Axioms

Theorem 4.1. *Let T be the set of tags in a tagspace and let $\mathcal{T}$ be the collection of upward-closed subsets under $\sqsubseteq$. Then $(T, \mathcal{T})$ is a topological space.*

Proof. We verify the three axioms.

(i) $\emptyset \in \mathcal{T}$ and $T \in \mathcal{T}$. The empty set is vacuously upward-closed. The full set T is upward-closed because there is no $b \in T$ outside T. Both belong to $\mathcal{T}$.

(ii) Finite intersections. Let $U, V \in \mathcal{T}$. Suppose $a \in U \cap V$ and $a \sqsubseteq b$. Then $b \in U$ (since U is upward-closed) and $b \in V$ (since V is upward-closed), so $b \in U \cap V$. Hence $U \cap V \in \mathcal{T}$.

(iii) Arbitrary unions. Let $\{U_\alpha\}$ be any collection of sets in $\mathcal{T}$. Suppose $a \in \bigcup_\alpha U_\alpha$ and $a \sqsubseteq b$. Then $a \in U_\alpha$ for some α. Since U_α is upward-closed, $b \in U_\alpha \subseteq \bigcup_\alpha U_\alpha$. Hence $\bigcup_\alpha U_\alpha \in \mathcal{T}$. □

The intersection argument also holds for arbitrary families, establishing the Alexandrov property. Therefore $(T, \mathcal{T})$ is a **topological space**. The following excerpt from `merge_containing_tags` illustrates the comparable-root cases of subtree intersection: containment of their roots identifies which subtree to retain. Representing subtrees by their roots connects the set-theoretic operation to the rank-prefix test.

```
if (b->rank().contains(a->rank())) {
    merged++;
} else if (a->rank().contains(b->rank())) {
    A.insert(*b);
}
```

Listing 9: Intersection of subtrees in tagd.cc

4.3 Specialization Order

Every topological space carries a **specialization preorder**. We use the convention $a \leq_{\text{sp}} b$ if $a \in \text{cl}(\{b\})$.

Proposition 4.1. *In the tagspace topology, $a \leq_{\text{sp}} b$ if and only if $a \sqsubseteq b$: the singleton closure consists of the tag and its ancestors.*

Thus `animal` $\leq_{\text{sp}}$ `mammal` $\leq_{\text{sp}}$ `dog`. Topological specialization in this convention runs opposite to taxonomic specialization: the closure of a specific tag contains its more general ancestors.

4.4 Continuity as Order-Preservation

Definition 4.2 (Tagspace Map)**.** *A map $f : T_1 \to T_2$ between two tagspaces is **topologically continuous** if inverse images of open sets are open: $f^{-1}(U) \in \mathcal{T}_1$ for every $U \in \mathcal{T}_2$.*

Proposition 4.2. *A map $f : T_1 \to T_2$ is continuous (with respect to the Alexandrov topologies) if and only if it is order-preserving: whenever $a \sqsubseteq b$ in T_1, we have $f(a) \sqsubseteq f(b)$ in T_2.*

This gives a precise mathematical definition of a **hierarchy-preserving map** between tagspaces. If `dog _is_a mammal` in T_1, a continuous map must send `dog` to a descendant of the image of `mammal` in T_2, possibly that image itself. Preserving labels, sub-relators, and predicates supplies additional conditions when a map is to preserve the full semantic structure.

5 The Hard Tag Base: A Common Subspace

5.1 Hard Tags

Tagspaces initialized with the same hard-tag definitions contain the same hard-tag hierarchy, defined in `hard-tags.h` [1]:

```
inline constexpr std::string_view HARD_TAG_ENTITY{"_entity"}; /// HARD_TAG_SUB
    HARD_TAG_ENTITY tagd::POS_TAG
inline constexpr std::string_view HARD_TAG_SUB{"_sub"}; /// HARD_TAG_SUB
    HARD_TAG_ENTITY tagd::POS_SUB_RELATOR
inline constexpr std::string_view HARD_TAG_IS_A{"_is_a"}; /// HARD_TAG_SUB
    HARD_TAG_SUB tagd::POS_SUB_RELATOR
inline constexpr std::string_view HARD_TAG_TYPE_OF{"_type_of"}; /// HARD_TAG_SUB
    HARD_TAG_SUB tagd::POS_SUB_RELATOR
inline constexpr std::string_view HARD_TAG_RELATOR{"_rel"}; /// HARD_TAG_SUB
    HARD_TAG_ENTITY tagd::POS_RELATOR
inline constexpr std::string_view HARD_TAG_HAS{"_has"}; /// HARD_TAG_SUB
    HARD_TAG_RELATOR tagd::POS_RELATOR
inline constexpr std::string_view HARD_TAG_CAN{"_can"}; /// HARD_TAG_SUB
    HARD_TAG_RELATOR tagd::POS_RELATOR
inline constexpr std::string_view HARD_TAG_TRUE{"_true"}; /// HARD_TAG_SUB
    HARD_TAG_ENTITY tagd::POS_TRUE
inline constexpr std::string_view HARD_TAG_FALSE{"_false"}; /// HARD_TAG_SUB
    HARD_TAG_ENTITY tagd::POS_FALSE
inline constexpr std::string_view HARD_TAG_INTERROGATOR{"_interrogator"}; ///
    HARD_TAG_SUB HARD_TAG_ENTITY tagd::POS_INTERROGATOR
```

Listing 10: Core hard tag definitions in hard-tags.h

5.2 The Hard Tag Subspace

Definition 5.1 (Hard Tag Subspace)**.** *Let $H \subseteq T$ denote the set of hard tags. For tagspaces using the same hard-tag definitions, the induced subspace topology on H is the same, since its restricted ancestry order is the same. Thus H is a common subspace shared by these tagspaces.*

Remark 5.1. *One can define a category of extensions of H: objects are order embeddings $i_T : H \hookrightarrow T$, and morphisms are monotone maps $f : T \to T'$ satisfying $f \circ i_T = i_{T'}$. In this category, the object $H \xrightarrow{\mathrm{id}} H$ is **initial**: it has a unique morphism to each object.*

5.3 Interoperability via the Common Base

Each tagspace is a world of discourse. When these worlds disagree about user-defined tags, the same version of the hard-tag hierarchy H supplies common ground for exchanging and interpreting TAGL.

After identifying their copies of H, and any further tags whose identities and structure agree, two tagspaces satisfy

$$H \subseteq T_1 \cap T_2.$$

If they share no user-defined tags, then $T_1 \cap T_2 = H$. Such tagspaces need not be laminar: $T_1 = H \cup \{a\}$ and $T_2 = H \cup \{b\}$, with distinct user tags a, b, overlap without either containing the other. Laminarity applies to descendant subtrees within one tree. Equal labels alone do not establish agreement between independently defined user tags.

> **Future development**
>
> For extensions in which H contains every ancestor of its tags, gluing only the copies of H gives a **categorical pushout** $T_1 \amalg_H T_2$ in posets with monotone maps. Keep the user tags distinct and inherit each tree's parent relation. The result is again a rooted tree: parents agree on H, and every other tag retains its original parent.

Any two monotone maps agreeing on H extend uniquely to this union, which is the pushout's universal property. The Alexandrov functor of Section 8.1, being a left adjoint, carries this pushout to topological spaces. This supplies a precise model for combining worlds; reconciling user labels, predicates, and rank assignments remains part of the merge design.

Implementation note

The hard-tag hierarchy provides the shared vocabulary used by `httagd`, the embedded HTTP application server having a Web API which maps to `TAGL` - *the language is the service*. Hard-tag definitions, including ranks, can change between versions; the signed versioning scheme in Section 9.5 would make agreement on a particular base explicit.

```
-- shared by tagspaces using the same hard-tag version:
_entity _sub _entity

-- user-defined, varies by tagspace:
>> dog _is_a mammal;
```

Listing 11: A shared hard-tag version; distinct user extensions

6 Predicates: Fibered Structure

6.1 Relators Are Tags

Predicate relators are themselves tags — subordinate to `_rel` in the tagspace tree. This means predicates are not outside the system; they are *inside* it. The tagspace is self-describing.

For example, predicate relators occupy related positions in the hierarchy [10]:

```
_rel     rank: 2        (root of predicate relators)
_has     rank: 2.1
_can     rank: 2.2
_refers  rank: 2.3
```

The language used to attach meaning to tags is itself expressed in the same structural language.

6.2 Predicates as Horizontal Relations

Predicates are relations attached to a tag that do not alter its position in the tree. TAGL groups objects under a relator; each attached predicate records one relator, one object, and optional modifier information [7, 3].

```
>> dog _is_a mammal     -- vertical: places dog in the tree
    _has legs = 4, tail
    _can bark;          -- horizontal: predicate data at dog
```

Listing 12: Vertical identity and horizontal predicates

6.3 The Two-Layer Structure

The tagspace carries two distinct mathematical layers:

Definition 6.1 (Two-Layer Tagspace Structure)**.** *1.* ***Base space****: the topological space $(T, \mathcal{T})$ defined by subordinate relations and ranks — the vertical dimension.*

2. ***Fiber****: at each tag $t \in T$, a set of predicates $P(t) \subseteq \mathrm{Rel} \times T \times \mathrm{Mod}$ — the horizontal dimension, where* Rel *is the set of relator tags and* Mod *is the set of modifier values.*

Remark 6.1 (Predicate fibers)**.** *Let $P(t)$ be the predicate set attached to tag $t \in T$. Form the disjoint union of these sets and project each tagged predicate onto its tag:*

$$E = \coprod_{t \in T} P(t) = \{(t, q) \mid t \in T,\ q \in P(t)\}, \qquad \pi : E \to T, \quad \pi(t, q) = t.$$

The fiber over t is

$$E_t = \pi^{-1}(\{t\}) = \{(t, q) \mid q \in P(t)\} \cong P(t).$$

*Thus each predicate set is a **fiber** over its tag, in the precise set-theoretic sense.*

The projection π brings the two layers together: the base space records hierarchical position, and the fiber records the predicates attached there.

Future development

This suggests a connection with **fiber bundles**. A topological bundle structure requires a topology on E, continuity of π, and local trivializations $\pi^{-1}(U) \cong U \times F$ respecting projection. Ordering predicates by rank can support indexing; establishing these bundle conditions is a separate mathematical construction.

Implementation note

The `predicate_set` iteration order uses the relator's rank, then the object's rank, then the modifier's literal UTF-8 bytes; an absent modifier precedes a present value. Operator and modifier-type metadata break otherwise equal keys. Canonical ranks are resolved before contextual referent encoding and retained privately by returned snapshot values. Comparators perform no storage access. Numeric membership equivalence remains separate from this literal ordering [4, 3]. This aligns predicate iteration with the hierarchy. For Alexandrov topologies on E and T, continuity of π additionally requires an order on E for which $(t, q) \leq_E (t', q')$ implies $t \sqsubseteq t'$; sorting alone does not establish this condition or local triviality.

The `abstract_tag` class in `tagd.h` encodes exactly this decomposition:

```
class abstract_tag {
    tagd::rank    _rank;       // position in the base space
    predicate_set relations;   // fiber: additional data at this point
};
```

Listing 13: Schematic two-layer structure in abstract_tag (tagd.h)

Implementation note

The predicate tuple above suppresses the operator and modifier-type fields of `predicate`, and allows an absent modifier. These fields remain part of the stored C++ predicate value.

6.4 Query Semantics as Selection of Sets

Queries select sets of tags through hierarchy and predicates. A subtree constraint at x selects from $D(x)$, while predicate conditions inspect the attached sets $P(t)$. A predicate-filtered result can be any subset of the selected subtree; it need not itself be open or closed.

Implementation note

The query interface also distinguishes immediate children from whole subtrees. A query with only a subordinate relation selects immediate children; predicate queries use rank-prefix matching and combine results through containment [5, 4].

For a concrete example, the supplied animal hierarchy defines `dog`, `cat`, `whale`, and `bat` beneath `mammal`, attaches `_has blood` to `mammal`, and `can bark` to `dog` [10]:

```
?? _what _sub mammal;          -- dog, cat, whale, bat
?? _what _has blood;           -- mammal
?? _what can bark;             -- dog
?? _what _has blood can bark;  -- dog
```

Listing 14: Hierarchy and predicate selection

> **Implementation note**
>
> Here `can` is the example's user-defined relator. The final query retains `dog`, the more specific root within the matching `mammal` subtree. This illustrates how rank containment combines hierarchical selections, beyond literal intersection of returned labels.

7 Structural Identity

7.1 Identity Is Position

The TAGL specification distinguishes nominal identity (the label) from structural identity (the subordinate relation and rank) [7].

In tagd, meaning is defined through relations. The string `dog` provides a name; its relations to `mammal`, `animal`, and ultimately `_entity` establish its place in the tagspace. Its predicates express how it relates to other tags.

Moving the tag changes its structural identity while its nominal label can remain the same. This distinction makes the maxim precise: *to be, is to be related.* A label locates the tag; its position and relations express what it means within the system.

7.2 The Canonical Form

Canonical serialization expresses the tagspace as TAGL text. From the tagd Design Thesis [8]:

> The canonical form of a tagspace is its dump — the UTF-8 TAGL code that represents the tagspace (and can be loaded).

The defining requirement for canonical serialization is the round-trip law

$$\text{load}(\text{dump}(T)) \cong T,$$

where the isomorphism preserves labels, subordinate relations, and predicates. The text then represents the same relational structure in a form that can be stored, exchanged, and reconstructed.

> **Future development**
>
> Completing this law for `dump()` is a planned development (Section 9). Cryptographic versioning additionally requires a deterministic encoding of all committed fields, so equivalent inputs produce identical bytes (Section 9.5).

8 Deeper Structures

The preceding sections connect tags, ranks, and predicates to familiar mathematical structures. These connections extend naturally into category theory and logic, providing a vocabulary for reasoning about the system and developing it further. Category theory, introduced by Samuel Eilenberg and Saunders Mac Lane [16], supplies the language of functors, adjunctions, and universal constructions used here.

8.1 The Adjunction Between Order and Topology

The relationship between the tagspace as a partial order and the tagspace as a topological space is not incidental. It is a formal **adjunction** — one of the most fundamental relationships in category theory.

Two functors mediate between the two views:

- $A : \mathbf{PreOrd} \to \mathbf{Top}$ (the *Alexandrov functor*) takes the rank prefix partial order on tags and produces the Alexandrov topological space described in Section 4.
- $S : \mathbf{Top} \to \mathbf{PreOrd}$ (the *specialization functor*) takes any topological space and recovers a preorder: $a \leq b$ iff every open set containing a also contains b.

$$\mathbf{PreOrd} \underset{S}{\overset{A}{\underset{\longleftarrow}{\overset{\longrightarrow}{\bot}}}} \mathbf{Top}$$

These functors form an adjunction: A is left adjoint to S, written $A \dashv S$. Applying both in sequence recovers the original structure: $S(A(P)) \cong P$.

The unit $P \to S(A(P))$ is the identity on underlying points: passing from the order to its topology and back recovers the same order. For tagd, this explains why rank-prefix containment and the neighbourhood structure are two descriptions of the same hierarchy.

8.2 Topological Properties of the Tagspace

The Alexandrov topology gives the tagspace several useful local properties.

First countable. Every tag x has a smallest open neighbourhood: the set $N(x)$ of descendants of x, including x itself. The single set $N(x)$ forms a local base at x.

Locally compact. For any tag x, the smallest open neighbourhood $U(x)$ is compact in the Alexandrov topology. Any open cover of $U(x)$ must include a member containing x, which then contains all of $U(x)$, giving a one-member subcover. This proves compactness of the smallest open neighbourhood, without a Hausdorff assumption.

Locally path connected. For $y \in N(x)$, the map $\gamma : [0,1] \to N(x)$ with $\gamma(0) = x$ and $\gamma(t) = y$ for $t > 0$ is continuous: the inverse image of an open set is empty, $(0,1]$, or $[0,1]$, each open in $[0,1]$. Hence each $N(x)$ is path connected, so these smallest neighbourhoods give local path connectedness.

Subspaces and finite products are Alexandrov. Any subset of tags — for example, the result of a `query()` or all descendants of a given tag — inherits an Alexandrov subspace topology. Subsets can therefore be studied with the order and topology restricted to their own tags.

8.3 Modal Operators: Interior and Closure

In any topological space, the **interior** and **closure** operators are well-defined. In the tagspace, they carry semantic meaning.

Topological operator	**Modal logic**	**tagd meaning**
Interior $\mathrm{int}(U)$	Necessarily ($\Box$)	Properties true at a tag and all its descendants
Closure $\mathrm{cl}(U)$	Possibly ($\Diamond$)	Properties true at a tag or some descendant

The power set $\mathcal{P}(T)$, with this interior operator, is an **interior algebra**. Interior and closure give a logical reading of the neighbourhood structure.

Furthermore, because the Alexandrov topology arises from a partial order, the tagspace constitutes a **Kripke frame**, in Saul Kripke's relational semantics: a set of worlds (tags) equipped with an accessibility relation (the ancestor-first order). A predicate P is *necessarily true* at tag x (written $\Box P(x)$) if P holds at every tag in $D(x)$. It is *possibly true* ($\Diamond P(x)$) if P holds at some tag in $D(x)$.

Reflexivity and transitivity make this an **S4 frame**, validating the modal system S4 of C. I. Lewis [15]. In particular,

$$\mathbf{T}: \quad \Box P \to P, \qquad \mathbf{4}: \quad \Box P \to \Box\Box P.$$

Axiom T holds because $x \in D(x)$; axiom 4 holds because $y \in D(x)$ implies $D(y) \subseteq D(x)$. Thus necessity includes the tag itself and propagates to its descendants. A particular tagspace may validate additional formulas beyond S4.

```
-- "dog necessarily _has legs" if dog and all descendants of dog _has legs
-- "dog possibly _can bark" if dog or some descendant of dog _can bark
```

Listing 15: Modal reading of TAGL predicates

Future development

This supplies the semantics for a future modal query evaluator (Section 9).

Remark 8.1 (Modal direction)**.** *For a valuation $V \subseteq T$ and accessibility $xRy \iff x \sqsubseteq y$,*

$$\Box V = \{x : N(x) \subseteq V\} = \mathrm{int}(V), \qquad \Diamond V = \{x : N(x) \cap V \neq \varnothing\} = \mathrm{cl}(V).$$

Both modalities use the same accessibility direction. Existential search among ancestors instead uses the converse relation and belongs to the dual-topology reading.

8.4 Constructive Existence: Tags as Witnesses

In **constructive mathematics**, existence is established by supplying a witness and evidence for its defining property. The **Brouwer–Heyting–Kolmogorov (BHK) interpretation**, named for L. E. J. Brouwer, Arend Heyting, and Andrey Kolmogorov, gives this reading to intuitionistic logic [13]. The **Curry–Howard correspondence**, associated with Haskell B. Curry and William A. Howard, expresses propositions as types and proofs as terms (programs). Per Martin-Löf's **intuitionistic type theory (MLTT)** makes the existential reading explicit through a dependent sum, or Σ-type [14]:

$$t : T, \quad e : Q(t) \quad \Longrightarrow \quad (t, e) : \sum_{x:T} Q(x).$$

Here $Q(x)$ is the type of evidence for a specified property of x. The tag t is the **witness**; the pair (t, e) is the constructed proof of $\exists x : T.\, Q(x)$.

For example, let $Q(x)$ express that x has the recorded subordinate relation `_is_a mammal`. Checking that relation for `dog` supplies evidence for this structural claim, making `dog` its existential witness. The evidence establishes the relation within the tagspace; further interpretations of `_is_a` require their own rules.

Future development

A proof-carrying formalization can make this evidence explicit as a term $e : Q(t)$ and certify the tagspace invariants. Its typing and validation rules would connect TAGL constructions to the established BHK and Curry–Howard interpretations (Section 9.3).

9 Planned Future Developments

The mathematical structure provides a foundation for further implementation and exploration. The developments below extend that foundation.

9.1 Completing the Structural Operations

Future development

Mutation validation must preserve the rooted tree and its rank encoding. Consistent root handling across containment and sorting, together with lossless, dependency-ordered serialization, completes the correspondence between these operations and the mathematical model.

Explicit numeric hard tags (`_number`, `_integer`, and `_float`) and logical distinctions between instance and type subordination are further developments. Numeric modifiers and the existing sub-relators already provide their starting points.

9.2 Kripke Semantics and Modal Queries

Future development

An evaluator for $\Box$ and $\Diamond$ would make the semantics of Section 8.3 directly available to TAGL queries and guards. It would evaluate necessity and possibility over the descendant neighbourhood $D(x)$, connecting logical assertions to rank containment.

9.3 Martin-Löf Type Theory and Dependent Structure

Future development

The predicate sets $P(t)$ form an indexed family over T. Their disjoint union has the structure of a dependent sum:

$$\sum_{t \in T} P(t) = \{(t, q) : t \in T,\ q \in P(t)\}.$$

This is the set-theoretic model of a Σ-type in MLTT [14]: (t, q) records a tag and attached predicate data. A proof-carrying version instead pairs t with evidence $e : Q(t)$ for a specified proposition, as in Section 8.4. Distinguishing predicate data $P(t)$ from proof types $Q(t)$ makes the logical claim precise.

The complementary construction is the **dependent product**, or Π-type: a term $f : \prod_{t:T} Q(t)$ supplies evidence $f(t) : Q(t)$ for every tag, interpreting universal quantification. When the result type is constant, this specializes to an ordinary function type. These established constructions provide a foundation for verified TAGL functions; realizing it requires typing, equality, and computation rules with total, proof-preserving evaluation. The rank function remains the string-valued encoding of position; the indexed families supply the dependent structure.

9.4 Computation as Topological Traversal

Future development

The TAGL specification and design thesis propose functions as tags, single-assignment variables, modal guards, and asynchronous processes [7, 8]. These planned language features extend the relational model from knowledge representation toward computation.

```
>> greet($name) _is_a function
    -> "Hello, " ++ $name;
```

Listing 16: Proposed function syntax

A function represented as a tag can participate in the same hierarchy and predicate relations as other tags. Modal guards can express conditions on its neighbourhood, while TAGL serialization provides a common language for exchanging structures between processes.

Under the MLTT interpretation, total functions with dependent result types inhabit Π-types (Section 9.3); the proposed runtime needs corresponding typing and evaluation rules to support this proof interpretation.

The topological question is then precise: which computations preserve the ancestry order? Whenever a computation defines an order-preserving map between tagspaces, it is continuous. Establishing such maps would connect execution to the topology already present in the representation.

9.5 Signed Merkle Versions of Tagspaces

Future development

A finite tagspace tree supports a Merkle construction by recursion from leaves to root [11]. Let $L(t)$ encode a tag's label, rank, subordinate relation, and complete predicate data. If $c_1, \ldots, c_k$ are its children (its upper covers), ordered lexicographically by rank, set

$$h(t) = \mathrm{Hash}(\mathrm{Enc}(\texttt{tagd-node-v1}, L(t), [h(c_1), \ldots, h(c_k)])) .$$

Here Enc is an unambiguous canonical encoding with field boundaries and deterministic ordering of predicate sets; leaves use an empty child list. Predicate references are encoded as data, so recursion follows only the tree.

Finiteness makes this definition well-founded. With a collision-resistant hash, the root digest commits to the encoded hierarchy and predicates. This is an exact representation commitment: labels and ranks matter, even when two underlying topologies are isomorphic.
Compute a separate digest h_H from the hard-tag base alone, excluding user extensions, and $h_T = h(\texttt{_entity})$ from the full tagspace. A signed manifest can bind the version, encoding and hash algorithm, h_H, and h_T. Verification under a trusted public key authenticates that versioned commitment [12]; matching base digests then establish the common H assumed in Section 5.3, under the collision-resistance assumption. Agreement on this base supports interoperability while allowing user-defined worlds to differ.

10 Summary

Table 1 brings together the mathematical structures and their roles in tagd.

Structure	**tagd correspondence**	**Role**
Rooted tree / prefix poset	Parent fields and unique ranks	Structural identity and containment
Lexicographic total order	Rank byte comparison; SQL binary collation	Contiguous ordering of subtrees
Alexandrov topology	Descendant-closed open sets	Neighbourhoods determined by containment
Common subspace	Hard-tag hierarchy H	Shared relational vocabulary
Predicate fibers	Sets $P(t)$ and their disjoint union	Horizontal relations over the hierarchy
Continuous maps	Monotone maps of tagspace posets	Preservation of ancestry
Adjunction $A \dashv S$	Passage between preorders and spaces	Recovery of order from topology
Local topological properties	Smallest open neighbourhoods $D(x)$	Local bases, compactness, and paths
Interior algebra / S4 Kripke frame	$\mathcal{P}(T)$ and accessibility $\sqsubseteq$	Modal semantics; evaluator planned
Canonical serialization	Lossless, dependency-safe round trips	Reconstruction of structure; completion planned
Dependent types / proof witnesses	Indexed predicate families and invariant proofs	Future formalization
Functions, guards, processes	Proposed TAGL runtime	Planned language developments
Signed Merkle versions	Canonical node hashes and signed base commitments	Planned verification of shared versions

Table 1: Mathematical structures and their roles in tagd.

11 Conclusion

The mathematics of tagd gives form to its central principle: *to be, is to be related*. Tags acquire structural identity through their place in a hierarchy and meaning through their relations. Rank prefixes encode that hierarchy as a partial order, with `_entity` as its least element. The associated Alexandrov topology makes subtrees open neighbourhoods and ancestry the closure of a tag.

Predicates supply the second layer: an indexed family of fibers over this space. Together, hierarchy and predicates express knowledge as relational structure. Order, topology, and logic offer complementary ways to reason about it. The proposed developments in modal queries, formal verification, and computation build on this foundation.

References

[1] tagd source code, `tagd/include/tagd/hard-tags.h`. Hard tag definitions and axioms.

[2] tagd source code, `tagd/include/tagd/rank.h`. Rank structure, prefix order, and `contains()` implementation.

[3] tagd source code, `tagd/include/tagd.h`. `abstract_tag`, `predicate_set`, and tag set operations.

[4] tagd source code, `tagd/src/tagd.cc`. `merge_containing_tags` and merge operations.

[5] tagd source code, `tagspace/tagdb/sqlite/src/sqlite.cc`. Schema, rank allocation, queries, mutation, and serialization.

[6] Mathlib, *Upper and lower sets topologies*. `https://leanprover-community.github.io/mathlib4_docs/Mathlib/Topology/Order/UpperLowerSetTopology.html`. Upper-set topology, singleton closure, and monotone continuous maps.

[7] *TAGL Specification*, `docs/TAGL-spec.md`. Language specification and semantic model.

[8] *tagd Design Thesis*, `docs/tagd-thesis.md`. Language design document including function model and computation semantics.

[9] *tagd README*, `README.md`. Working examples and TAGL tutorial.

[10] *Example tagspace and rank listing*. The supplied animal hierarchy is defined in `tagsh/tests/bootstrap.tagl`. Its `.dump_grid` listing shows each tag's parent and rank. Generate it with:

```
cd tagsh/
bin/tagsh --file tests/bootstrap.tagl --tagl '.dump_grid' --noshell
```

[11] B. Laurie et al., *Certificate Transparency Version 2.0*, RFC 9162, Section 2.1 (Merkle trees). `https://www.rfc-editor.org/rfc/rfc9162.html`.

[12] NIST, *Digital Signature Standard (DSS)*, FIPS 186-5, 2023. `https://doi.org/10.6028/NIST.FIPS.186-5`.

[13] Philip Wadler, *Propositions as Types*. Communications of the ACM 58(12), 75–84, 2015. `https://doi.org/10.1145/2699407`. BHK interpretation and the Curry–Howard correspondence.

[14] Per Martin-Löf, *Intuitionistic Type Theory*. Notes by Giovanni Sambin, Bibliopolis, 1984. `https://archive-pml.github.io/martin-lof/pdfs/Bibliopolis-Book-retypeset-1984.pdf`. Dependent sums, dependent products, and their logical interpretations.

[15] André Platzer, *Classical Modal Logic*. 15-816 lecture notes, Carnegie Mellon University, 2010. `https://www.cs.cmu.edu/~fp/courses/15816-s10/lectures/05-pml.pdf`. S4 axiomatics and historical references to Lewis and Kripke.

[16] Samuel Eilenberg and Saunders Mac Lane, *General Theory of Natural Equivalences*. Transactions of the American Mathematical Society 58(2), 231–294, 1945. `https://doi.org/10.1090/S0002-9947-1945-0013131-6`.